



MULTITHREADING

PROGRAM

- A **program** is a set of instructions written in a programming language that tells a computer what to do.
- Is not running by itself; it is simply stored in a file (like .py, .exe, .java, .c, .cpp, etc.)
- Contains a sequence of instructions that describe how to perform a task
- Is written by a programmer or developer
- Needs to be loaded into memory and executed to become a process
- Programs are stored in **secondary memory** (like Hard Disk, SSD, Pen drive).

PROCESS

- A **process** is a **program in execution**, meaning it is the active running state of a program after the operating system loads it into memory and assigns resources to it.
- A **program** becomes a **process** only when it starts running.
- When a process is created, it gets:
 - ✓ Memory space (RAM)
 - ✓ CPU time
 - ✓ Process ID (PID)
 - ✓ Input/Output resources
 - ✓ Files and system resources

So, a process = **program + resources + execution state**.

LIFE CYCLE OF A PROCESS

State	Meaning
New	Process is being created
Ready	Waiting to be assigned to CPU
Running	CPU is executing the process
Blocked/Waiting	Process is waiting for I/O (ex: file, input)
Terminated	Process has finished execution

TYPES OF PROCESSES

Type	Example
Foreground / Interactive	Web browser, media player
Background	Antivirus scan, system update
User Process	Apps installed by user
System Process	OS-level tasks (e.g., task scheduler)

THREAD

- A **thread** is the **smallest unit of execution within a process**.
- It represents a **single sequence of instructions** that the CPU can execute independently.
- A **process** can contain **one or multiple threads**.
- Each thread shares the **same memory and resources of the process**, but runs **independently**.
- Example:

When you use a web browser:

One thread displays the web page

Another thread downloads files

Another thread plays video/audio

Process = Container, thread = Worker inside the container

FEATURES AND TYPES OF THREADS

- Features of a Thread

Feature	Explanation
Lightweight	Needs fewer resources than a process
Faster	Less overhead, quicker context switching
Shared data	Shares memory of the process
Independent	Executes parallel to other threads

- Types of Threads

Type	Meaning
User-level thread	Managed by user applications
Kernel-level thread	Managed by OS kernel

MULTITHREADING

- **Multithreading** is the ability of a process to run multiple threads concurrently so that different parts of a program can execute in parallel.
- **Multithreading** allows a program to run **multiple threads at the same time** within a **single process**.
- Each thread:
 - ✓ Runs independently
 - ✓ Shares the same memory and resources of the process
 - ✓ Performs different tasks in parallel

COMMON NETWORK PROTOCOLS

Protocol	Purpose
HTTP	Browsing websites
HTTPS	Secure website browsing
FTP	File transfer
SMTP	Sending emails
TCP	Reliable data transmission
IP	Identifying devices and addressing
DNS	Convert domain name to IP address

CONNECTION ORIENTED SERVICE

A **connection-oriented service** requires creating a **connection before sending data**.

Example : Like making a **phone call** — first you connect to the person, then you start talking.

How it works:

- ✓ Connection is established
- ✓ Data is transferred
- ✓ Connection is closed

Key features:

- ✓ communication
- ✓ Data arrives in order
- ✓ No data loss (if lost, it is retransmitted)
- ✓ Slower due to connection setup

Real protocol example : TCP

CONNECTIONLESS SERVICE

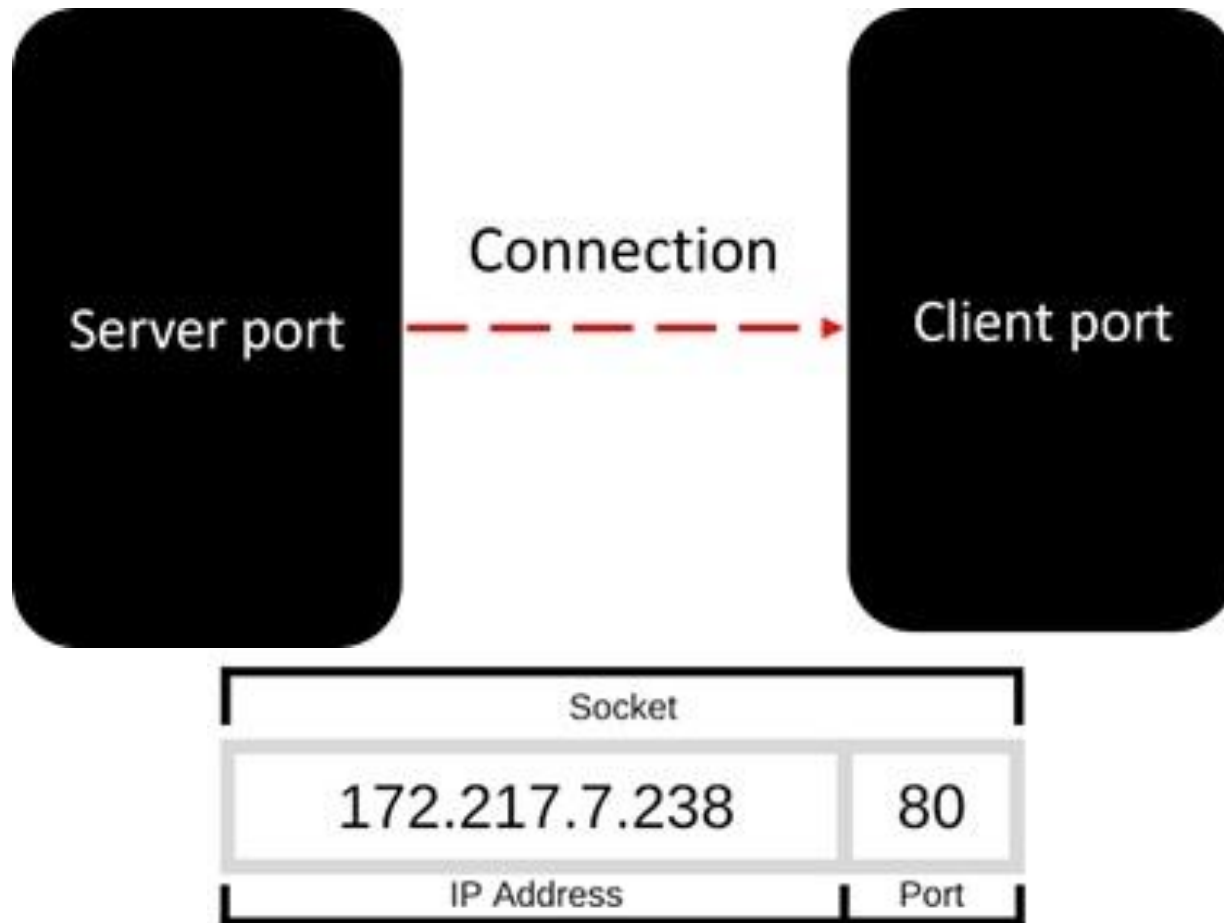
- A **connectionless service** sends data **without establishing a connection**.
- **Simple example** : Like **sending a text message or letter** — you just send it without checking if the receiver is ready.
- **How it works:**
 - ✓ Data is sent immediately — no connection setup
 - ✓ Receiver may or may not get it
- **Key features:**
 - ✓ Faster communication
 - ✓ Less reliable (data may be lost)
 - ✓ Data may arrive out of order

Real protocol example: **UDP (User Datagram Protocol)** is connectionless.

SOCKET

- A socket is the connection point that allows applications on different devices to communicate over a network.
- A socket is like a **door** in a computer through which data goes **in and out** when communicating with other devices.
- To send data over a network:
 - ✓ IP address tells **which device**
 - ✓ Port tells **which application**
 - ✓ **Socket** combines both to **create a communication link**
- A socket allows two programs to:
 - ✓ Open communication
 - ✓ Send data and Receive data

SOCKET





END..